



**Кафедра информационных систем
в химической технологии**

**Институт тонких химических технологий
им. М.В. Ломоносова
РТУ МИРЭА**



ПРОГРАММИРОВАНИЕ В ЗАДАЧАХ ХТ

Лекция 7. Функции и модули

Содержание лекции

В данной лекции будут рассмотрены следующие темы:

-  Определение функции
-  Применение функций
-  Создание функций
-  Область видимости переменных
-  Описание параметров функции
-  Возвращаемое функций значение
-  Анонимные и рекурсивные функции
-  Импортирование модулей
-  Запуск модуля в консоли

Определение функции

- 📡 Функция – специальным образом обозначенный участок кода, который вызывается по имени и выполняет определенную задачу
- 📡 Пользовательские функции создаются разработчиком
- 📡 Встроенные функции – часть языка программирования

Описание функции

```
def add(x, y):  
    return x + y
```

```
print(add(2, 3))
```

Вызов функции

Вывод: 5

Результат выполнения функции

Применение функций

- 📡 Организация часто повторяющихся операций
 - ⦿ Часто повторяющиеся операции выносятся в функции
- 📡 Снижение количества кода
 - ⦿ Функция создается один раз и вызывается при необходимости
- 📡 Повышение читаемости кода
 - ⦿ Использование функций структурирует код

```
def calc_circle(r):  
    circle_length = 2 * 3.14 * r  
    circle_area = 3.14 * r * r  
    return circle_length, circle_area  
  
calc_circle(3)
```

Создание функции

🌀 Для создания функции необходимо:

1. Использовать ключевое слово **def**
2. Указать имя функции
 - Маленькие буквы
 - Слова разделяются символом подчеркивания (_)
3. Определить параметры
 - Могут отсутствовать
4. Написать исполняемый функцией код

```
1      2      3
↓      ↓      ↓
def simple_function(param_1, param_2):
    result = param_1 - param_2
    return result
```

← 4

Обратите внимание

📡 Имена функций:

📡 example_function



📡 function

📡 example_function_1

📡 Example_Function



📡 FuNcTiOn

📡 examplefunction1

📡 Стил mixedCase можно использовать там, где он уже преобладает

📡 Двоеточие и табуляция:

```
def simple_function(param_1, param_2):
```

```
    result = param_1 - param_2
```

```
    return result
```

табуляция

← ДВОЕТОЧИЕ

Области видимости переменных

Область видимости

- ⦿ Определяет доступность переменной
- ⦿ Обращение к недоступному объекту вызывает ошибку `NameError`

В Python существуют 3 области видимости:

- ⦿ Локальная (`local`)
- ⦿ Глобальная (`global`)
- ⦿ Нелокальная (`nonlocal`)

Локальная область видимости (1 из 2)

- 📶 Области видимости «по умолчанию»
- 📶 Наиболее часто используемая область видимости
- 📶 Переменная попадает в ту область, в которой была создана
- 📶 Параметры функции попадают в локальную область видимости

```
x = 5
```

```
def func(x):  
    print(x)
```

```
func(10) Вывод: 10 – значение локальной переменной функции  
print(x) Вывод: 5 – значение глобальной переменной x
```


Локальная область видимости (2 из 2)

- 🔊 В функции доступны внешние переменные
- 🔊 Внутренняя для функции переменная переопределяет внешнюю

```
outside = 5
outside_2 = 2

def func(x):
    outside = x
    print(outside) Вывод: 10 – переопределенное значение
    print(outside2) Вывод: 2 – глобальная переменная outside2

func(10)
print(outside) Вывод: 5 – глобальная переменная outside
print(outside_2) Вывод: 10 – глобальная переменная outside2
```

Глобальная область видимости

- Используется оператор `global`
- Позволяет изменять значение «внешних» переменных
- Рекомендуется использовать только при необходимости

```
outside = 5
def func(x):
    global outside
    outside = x
```

```
func(10)
print(outside)
```

Вывод: 10 – измененное внутри функции значение глобальной переменной `outside`

Нелокальная область видимости

- Используется оператор `nonlocal`
- Даёт доступ к внешним для функции переменным, но не являющимися глобальными
- Чаще всего используется при определении одной функции внутри другой

```
num = 5
def counter():
    num = 0
    def incrementer():
        nonlocal num
        print(num += 1)
    incrementer()
counter()
```

Вывод: 1 – увеличенное на 1 значение нелокальной переменной num

Схема областей видимости

```
num = 10
```

```
sum = 5
```

```
sum = 3
```

```
global num
```

```
print(num) Вывод: 10
```

```
nonlocal sum
```

```
print(sum) Вывод: 3
```

Описание параметров функций

- 📡 Параметры функции – локальные переменные функции
 - ⦿ Значение присваивается при вызове функции
- 📡 Аргументы функции – конкретные значения параметров
 - ⦿ Передаются в функцию при вызове функции
- 📡 Часто параметры называют так же аргументами

```
def func(a, b, c):  
    ...  
func(1, 2, 3)
```

← параметры

← аргументы

Типы параметров

Позиционные параметры

- ⦿ При вызове функции значения присваиваются параметрам по порядку

Именованные параметры

- ⦿ При вызове функции значения присваиваются параметрам по именам, порядок не играет роли

При вызове функция можно использовать как позиционные, так и именованные параметры

```
def frac(num, den):  
    return num / den
```

```
res = frac(10, 5) # 2.0  
res = frac(5, 10) # 0.5  
res = frac(num=10, den=5) # 2.0  
res = frac(den=10, num=5) # 0.5  
res = frac(10, den=5) # 2.0
```

```
res = frac(10, numer=5)  
# ОШИБКА!  
# Два значения для numer
```

```
res = frac(denom=10, 5)  
# ОШИБКА!  
# Позиционные параметры после  
именных
```

Разделители параметров

- Разделители определяют, как разрешено присваивать значения параметрам
 - До '*' идут позиционные параметры
 - После '/' идут именованные параметры
 - Параметры между '*' и '/' можно задавать двумя способами

```
def func(a, b, *, c, d, /, e, f):  
    ...  
func(1, 2, 3, d=4, e=5, f=6) # ОК  
func(1, 2, 3, 4, 5, f=6) # ОШИБКА  
func(1, b=2, c=3, d=4, e=5, f=6) # ОШИБКА
```

Параметры со значением по умолчанию

Параметры могут быть необязательными

- ⦿ Для необязательных параметров указывается значение по умолчанию
- ⦿ Параметры со значениями по умолчанию должны идти после остальных параметров

```
def func(a, b, c=3):
```

← необязательный параметр

...

```
func(a=1, b=2)
```

```
Def func2(a, b=2, c): # ОШИБКА
```

...

Функции с переменным числом параметров

Произвольное количество позиционных параметров

- ⦿ Используется знак *
- ⦿ args – кортеж из переданных аргументов

```
def func(*args):
```

```
    print(args)
```

```
func(1, 2, 3, '123')
```

Вывод: (1, 2, 3, '123') ← кортеж

Произвольное количество именованных параметров

- ⦿ Используются два знака *
- ⦿ kwargs – словарь из переданных аргументов

```
def func(**kwargs):
```

```
    print(kwargs)
```

```
func(a=1, b=2, c=3)
```

Вывод: {'a': 1, 'b': 2, 'c': 3} ← словарь

Возвращаемое функцией значение

- Для возврата значения используется оператор **return**
- Функция может возвращать любые объекты:
 - Строки, числа, функции и т.д.
- Возвращает **None**, если **return** отсутствует
- Выполнение функции останавливается после первого **return**

```
def func(a):  
    if a > 5:  
        return a  
    return a + 5
```

```
result = func(6)  
print(result)
```

Вывод: 6 – сработал первый **return**, так как 6 больше 5.

Возвращаемое функцией значение

 Функция может возвращать несколько значений

- ⦿ Неявно используются кортежи

```
def calc_circle(r):  
    circle_length = 2 * 3.14 * r  
    circle_area = 3.14 * r * r  
    return circle_length, circle_area
```

```
length, area = calc_circle(3)
```

```
print(length)
```

Вывод: 18.84 – первое из возвращаемых значений

```
print(area)
```

Вывод: 28.26 – второе из возвращаемых значений

Анонимные функции (1 из 2)

- Анонимная функция (лямбда-функция) – функция, содержащая одно выражение, и не имеющая имени
- Для объявления используется ключевое слово **lambda**
- Описание параметров как в обычной функции
- Возвращается значение единственного выражения

анонимная функция

```
sum = lambda x, y: x + y
print(sum(2, 3))
```

↑
возвращаемое
значение

↓
параметры

обычная функция

```
def sum(x, y):
    return x + y
print(sum(2, 3))
```

Анонимные функции (2 из 2)

- 🔊 Анонимные функции часто используются во многих встроенных функциях
 - 🔊 map
 - 🔊 filter
 - 🔊 sorted
- 🔊 Анонимными функциями не стоит злоупотреблять
 - 🔊 Они удобны для разработки, но снижают читаемость кода

```
cars = ['vw', 'bwm', 'gaz']
```

```
list(map(lambda car: car.upper(), cars))
```

Вывод: ['VW', 'BWM', 'GAZ']

```
list(filter(lambda car: len(car) > 2, cars))
```

Вывод: ['bwm', 'gaz']

Рекурсивные функции

- 📡 Рекурсивная функция – функция, которая вызывает сама себя
- 📡 Примеры решаемых с помощью рекурсии задач:
 - 🕒 Числа Фибоначчи
 - 🕒 Факториал числа
- 📡 Важно задать условие выхода из рекурсии

```
def factorial(n):
```

```
    if n != 0:
```

```
        return n * factorial(n-1)
```

```
    else:
```

```
        return 1
```

← рекурсивный вызов

← выход из рекурсии

Алгоритм работы рекурсивной функции

```
def factorial(n):  
    if n != 0:  
        return n * factorial(n-1)  
    else:  
        return 1
```

$n = 4$

$\text{factorial}(4) = 4 * \text{factorial}(3)$

$\text{factorial}(3) = 3 * \text{factorial}(2)$

$\text{factorial}(2) = 2 * \text{factorial}(1)$

$\text{factorial}(1) = 1 * \text{factorial}(0)$

$\text{factorial}(0) = 1$

$\text{factorial}(1) = 1$

$\text{factorial}(2) = 2$

$\text{factorial}(3) = 6$

$\text{factorial}(4) = 24$

Функциональное программирование (1/2)

- 🌀 Функции высшего порядка принимают другие функции в качестве аргументов и/или возвращают другие функции
- 🌀 Примеры (встроенные функции):
 - 🌀 `filter()` – исключает элементы, не удовлетворяющие условию, заданному функцией
 - 🌀 `map()` – применяет функцию к каждому элементу
 - 🌀 `sorted()` – сортирует элементы по правилу, заданному функцией

```
def is_small(x):  
    return x < 4
```

```
l = [1,2,3,4,5]  
small = list(filter(is_small, l))  
print(small) # [1, 2, 3]
```

```
squares = list(map(lambda x : x^2, small))  
print(squares) # [1, 4, 9]
```


Функциональное программирование (2/2)

- При возврате функции в качестве результата вызова другой функции значения переменных остаются в ее *замыкании*

```
def power(exponent):  
    def raise_power(base):  
        return base ** exponent  
    return raise_power
```

```
square = power(2)  
cube = power(3)
```

```
print(square(4))    # 16 (42)  
print(cube(2))      # 8 (23)
```

Модуль Python

📡 Модуль – файл с расширением `.py`, содержащий код Python

📡 Модуль может содержать:

- 🕒 Функции
- 🕒 Переменные
- 🕒 Исполняемый код

📡 Пример модуля:

Для вывода содержимого модуля используйте функцию `dir(имя_модуля)`

```
# Модуль my_module.py
```

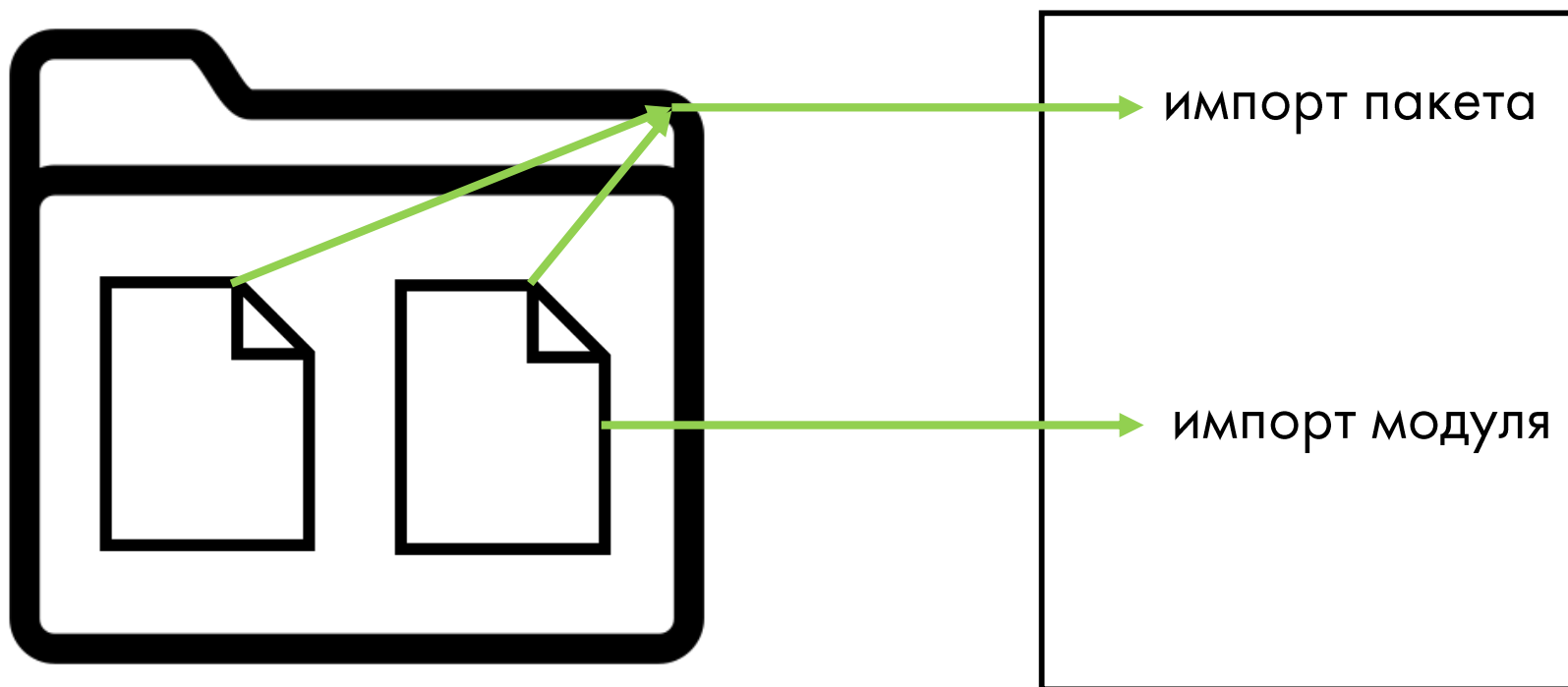
```
def func(a, b):  
    return a + b
```

```
var_1 = 1000
```

Содержимое данного модуля доступно для импорта

Импортирование модулей (1 из 6)

- 📁 Модуль в Python – любой файл с программой
- 📁 Пакет в Python – любая папка
- 📁 При импорте модуля выполняется весь его код
- 📁 Существуют пользовательские и встроенные модули



Импортирование модулей (2 из 6)

Используется оператор `import`

- После оператора перечисляются модули
 - Для встроенных модулей указывается только имя модуля
 - Для пользовательских – путь к файлу и имя файла без расширения
- После импортирования модуля его название становится переменной, через которую можно получить доступ к атрибутам модуля
 - Доступ к атрибутам через оператор «.» (точка)

```
import time, random
```

```
time.time()
```

```
# 1376047104.056417
```

```
random.random()
```

```
# 0.9874550833306869
```

```
import my_module
```

```
print(my_module.func(2,3))
```

↑
Обращение через имя
модуля, оператор и имя
атрибута

Импортирование модуля (3 из 6)

- Из модуля можно импортировать отдельные атрибуты
 - `from <module_name> import <attribute_name>`
- Можно импортировать сразу все атрибуты
 - `from <module_name> import *`
- Атрибутам и модулям можно назначать псевдонимы
 - `from <module_name> import <attribute_name> as <alias>`
 - `import <module_name> as <alias>`

```
from my_module import var_1
from my_module import *
```

```
print(var_1)           1000
print(first_var)       1000
```

```
from my_module import var_1 as v1
import my_module as my
```

```
print(v1)              1000
print(my.var_1)        1000
```

Импортирование модулей (4 из 6)

Импорт пакетов

- ⦿ Указывается путь к модулю в пакете
- ⦿ Доступ к модулям через «.»

Импорт отдельных модулей

- ⦿ Доступ по имени модуля
- ⦿ `from <pack_name> import <module_name>`

```
import pack_a.module_a  
print(pack_a.module_a.attr)
```

```
from pack_a import module_a  
print(module_a)
```

Импортирование модулей (5 из 6)

Тонкости импортирования:

- ⦿ Возможна перезапись переменных
- ⦿ Переменные существуют в разных контекстах

```
import math  
from math import e
```

```
e = 2 #Перезапись импортированного атрибута!  
print(e)
```

Вывод: 2 – перезаписанное значение из контекста программы

```
print(math.e)
```

Вывод: 2.7 – значение атрибута «e» из контекста модуля math

Импортирование модулей (6 из 6)

 `import` и `from` – исполняемые выражения

- ⦿ Выполняются по ходу исполнения кода

 Гибкий подход к импортированию

```
value = int(input())
```

```
if value>10:
```

```
    import math
```

```
else:
```

```
    import cmath
```


Импортирование модуля: ошибки

- Если импортируемый модуль или атрибут не был найден, то возникнет исключение **ModuleNotFoundError** или **ImportError**

```
Traceback (most recent call last):  
  File "D:/Работа/PYTHON/test/test2.py", line 1, in <module>  
    import test_error  
ModuleNotFoundError: No module named 'test_error'
```

- Если обратиться к атрибуту, которого нет в модуле, то возникнет ошибка **AttributeError**

```
Traceback (most recent call last):  
  File "D:/Работа/PYTHON/test/test2.py", line 3, in <module>  
    print(test_error.var_1)  
AttributeError: module 'test_error' has no attribute 'var_1'
```

Запуск модуля в консоли (1 из 2)

- При импорте весь код модуля выполняется
 - Исключением является часть кода, которая выполняется только при запуске в консоли
- Это даёт возможность использовать Python модули в качестве скриптов
 - Для выполнения модуля необходимо в консоли выполнить команду:
`python относительный_путь_к_файлу`

```
# Модуль my_module.py  
  
print('Hello world!')
```

Консоль открыта в
директории с модулем

- Пример запуска: `> python my_module.py`
- Вывод в консоли: Hello world!

Запуск модуля в консоли (2 из 2)

- Если один и тот же модуль импортируется и используется как скрипт, то удобно использовать следующую конструкцию:

```
if __name__ == "__main__":  
    # code
```

- Помещенный в данную конструкцию код будет исполнен только в случае запуска модуля в виде скрипта

- При импорте модуля такой код исполнен не будет

Код, исполняемый
запуске в консоли

```
def func(a, b):  
    return a + b  
  
if __name__ == "__main__":  
    print(func(2, 3))
```

Код, исполняемый
при импорте

Передача параметров

- 📡 Для ввода и вывода используются функции `input()` и `print()`
 - 🕒 `input()` ожидает пользовательского ввода
 - 🕒 `print()` выводит информацию в консоль
- 📡 При запуске модуля в качестве скрипта имеется возможность передавать параметры
 - 🕒 Значения параметров указываются через пробел после имени модуля
 - 🕒 Доступ к параметрам осуществляется через список `sys.argv` модуля `sys`

```
if __name__ == "__main__":  
    import sys  
    print(sys.argv)           ['my_module.py', '10', 'arg_2']  
    print(sys.argv[1])       '10'
```

- 📡 Пример: `python my_module.py 10 arg_2`

Пример

- Модуль `fibonacci.py` содержит функцию, вычисляющую n -го числа ряда Фибоначчи
- Данный модуль можно импортировать или запускать как скрипт

```
# Модуль fibonacci.py

def fib(n):
    if n in (1, 2): return 1
    return fib(n - 1) + fib(n - 2)

if __name__ == "__main__":
    import sys
    argument = int(sys.argv[1])
    print(fib(argument))
```

```
import fibonacci

print(fibonacci.fib(10))
Вывод: 55
```

```
> python fibonacci.py 10

Вывод: 55
```

Итоги

В данной лекции были рассмотрены следующие темы:

-  Определение функции
-  Применение функций
-  Создание функций
-  Область видимости переменных
-  Описание параметров функции
-  Возвращаемое функций значение
-  Анонимные и рекурсивные функции
-  Импортирование модулей
-  Запуск модуля в консоли